

5-Mark Questions & Answers

Q1. Explain the difference between File System and Database System with a diagram.

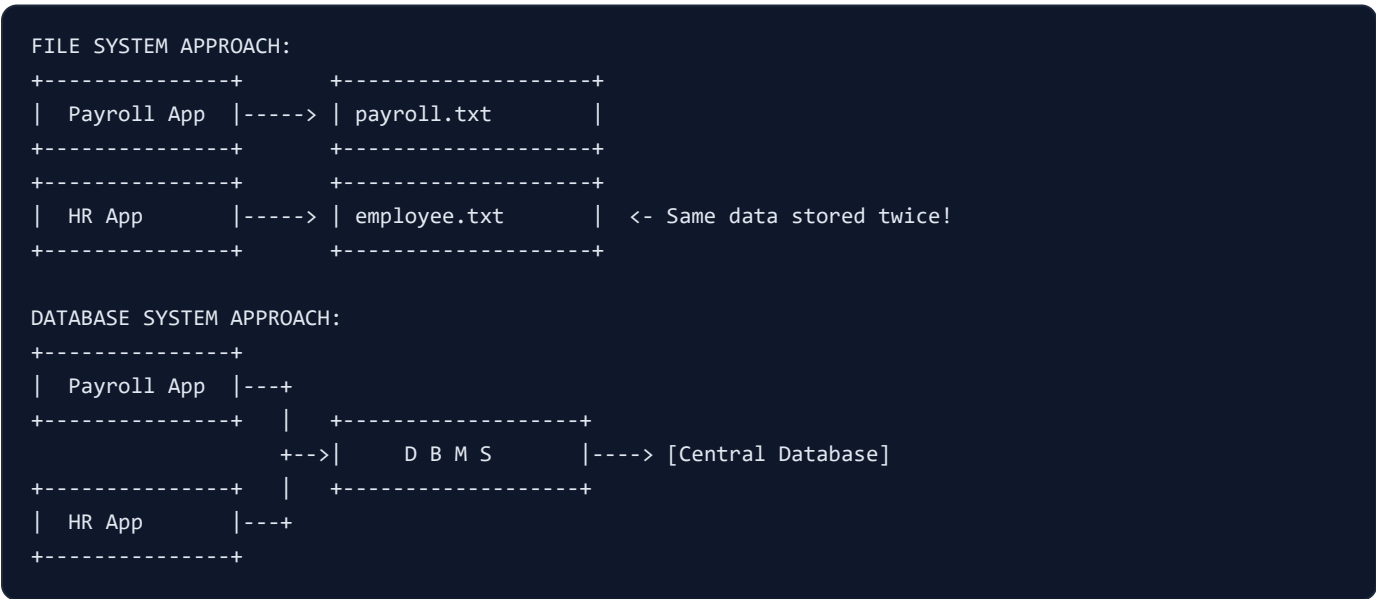
File System stores data in separate, unrelated files managed by the OS. Each application has its own set of files with no centralized control, leading to redundancy and inconsistency.

Database System centralizes all data under a DBMS, which enforces structure, integrity, and security across all applications.

Key Differences:

Feature	File System	Database System
Data Redundancy	Very High	Minimal (via normalization)
Data Consistency	Poor	High (single source of truth)
Security	OS-level only	Fine-grained (user/table/column level)
Concurrency	Not supported properly	Built-in concurrency control
Recovery	No automatic recovery	Transaction-based recovery (ACID)
Query Language	None (custom code required)	SQL
Data Sharing	Difficult	Easy and controlled

Diagram:



Q2. Explain the three-schema (three-level) architecture of DBMS with a neat diagram.

The **Three-Schema Architecture** (proposed by ANSI/SPARC) separates the database into three levels to achieve data abstraction and independence.

1. External Level (View Level):

The top-most level. Represents individual user views. Different users see only the data they are authorized to see. Multiple external views can exist simultaneously.

Example: HR sees salary data; Accounts sees payment records — both from the same database.

2. Conceptual Level (Logical Level):

The middle level. Describes the entire logical structure — all entities, attributes, relationships, and constraints — without any physical storage detail.

Example: Full schema with `Student(RollNo, Name, Branch)` and `Course(CourseID, Name)` tables.

3. Internal Level (Physical Level):

The lowest level. Describes how data is physically stored — file organization, indexes, data blocks, and access paths.

Diagram:

```
+-----+
| EXTERNAL LEVEL |
| [View 1] [View 2] [View 3] | <- Different user views
+-----+
| Mapping 1 |
+-----+
| CONCEPTUAL LEVEL |
| [Full logical schema of DB] | <- Complete logical design
+-----+
| Mapping 2 |
+-----+
| INTERNAL LEVEL |
| [Physical storage on disk] | <- Actual storage structures
+-----+
```

Q3. Explain Data Abstraction and its three levels in detail with a diagram.

Data Abstraction is the process of hiding irrelevant implementation details from users to reduce complexity and improve usability.

1. Physical Level:

Describes the *how* of storage — low-level data structures (B+ trees, hash files), storage allocation, and access paths. Managed entirely by the system; invisible to all users.

2. Logical Level:

Describes the *what* of data — what data is stored and the relationships between them. Implemented using DDL. DBA and programmers work at this level.

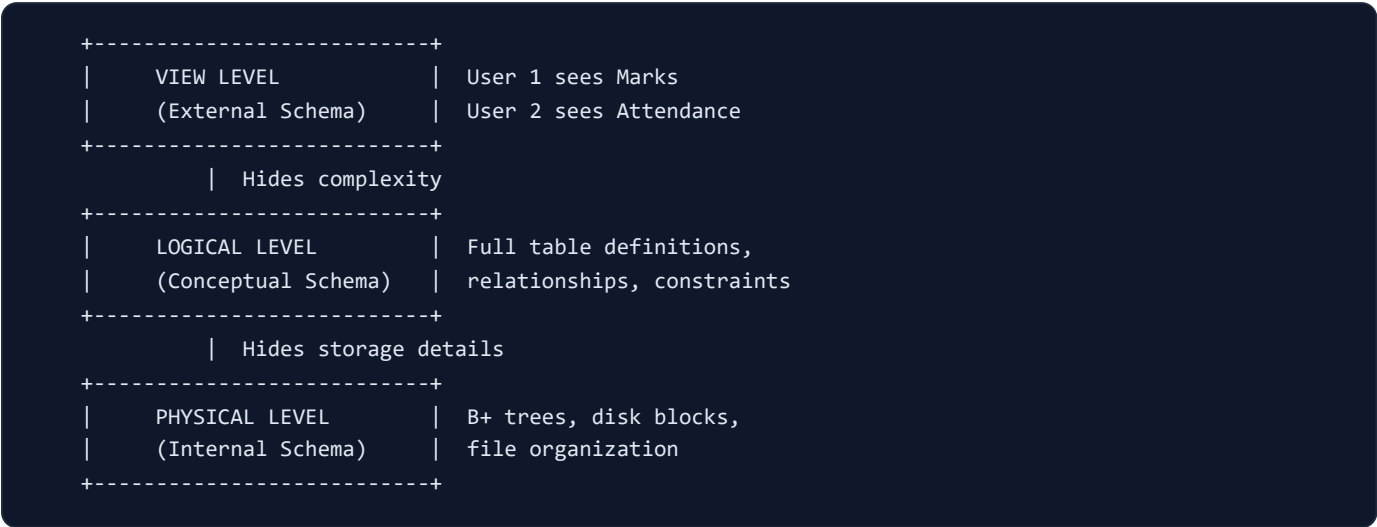
```
CREATE TABLE Student (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50),
    Branch VARCHAR(30)
);
```

3. View Level:

The highest and most user-friendly level. Each user sees a customized, partial view of the database, hiding the rest for

simplicity and security.
Example: A student views only their own marks, not others'.

Diagram:



Q4. Explain Data Independence and its two types in detail with a diagram.

Data Independence is the ability to change the schema at one level without requiring changes at the next higher level. It is a key advantage of the three-level architecture.

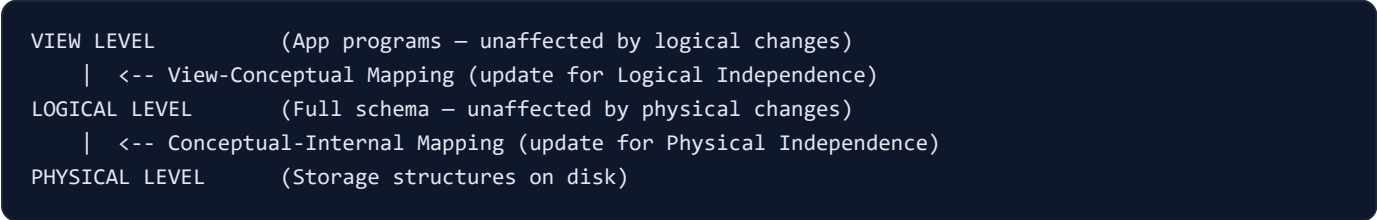
Type 1 — Logical Data Independence:

Ability to change the **conceptual schema** without affecting the **external views** or application programs.
Example: Adding a **Phone** column to the **Student** table should not break apps that only use **Name** and **RollNo**.
Achievement: Update only the **View-Conceptual Mapping**.

Type 2 — Physical Data Independence:

Ability to change the **internal/physical schema** without affecting the **conceptual schema**.
Example: Switching from heap file storage to B-tree indexing should not require rewriting the **CREATE TABLE** SQL.
Achievement: Update only the **Conceptual-Internal Mapping**.

Diagram:



	Logical Independence	Physical Independence
Schema Changed	Conceptual	Internal/Physical
Level Protected	External views	Conceptual
Difficulty	Harder	Easier

Q5. Describe the different types of Database Users with examples.

1. Naive Users:

End-users who interact through pre-built app interfaces (forms, menus). No DBMS knowledge.

Example: Bank customer checking balance via mobile app.

2. Application Programmers:

IT professionals who write programs in host languages (Java, Python) embedding DML calls. Their apps are used by naive users.

Example: Developer who builds the online banking app.

3. Sophisticated Users:

Interact directly with the DBMS using SQL or tools like Power BI, Tableau.

Example: Data analyst writing `SELECT * FROM Sales WHERE region = 'West' .`

4. Specialized Users:

Build specialized applications beyond the traditional relational framework — CAD, GIS, AI expert systems.

Example: A GIS developer using a spatial DBMS to build a map app.

5. Database Administrator (DBA):

Most privileged user. Controls the entire system — defines schema, grants access, performs backups, monitors performance.

Example: The IT professional who creates and manages the university's Student database.

Q6. Explain the role and responsibilities of a Database Administrator (DBA).

The **DBA** is the central authority of the database system with complete control and highest privilege level.

1. **Schema Definition:** Writes the original DDL to create the database schema — tables, views, indexes, and integrity constraints.
 2. **Storage Structure and Access Method Definition:** Defines physical data organization on disk. Chooses appropriate file organizations and indexing methods for optimal performance.
 3. **Schema and Physical Organization Modification:** Alters the schema when business requirements change or modifies physical storage to improve performance.
 4. **Granting User Authorizations:** Creates user accounts and uses `GRANT` / `REVOKE` to manage which users can perform which operations on which data.
 5. **Routine Maintenance:** Performs periodic data backups, monitors disk space and query performance, applies software patches, and ensures high availability.
 6. **Performance Tuning:** Analyzes slow queries, creates indexes, and restructures tables to ensure fast response times.
 7. **Defining Integrity Constraints:** Adds rules like `NOT NULL` , `UNIQUE` , `CHECK` , `FOREIGN KEY` to maintain data accuracy and consistency.
-

Q7. Explain the components of the Storage Manager in detail.

The **Storage Manager** is the interface between the higher-level DBMS and physical data stored on disk.

Components:

1. **Authorization and Integrity Manager:** Checks user privileges before executing any query. Also enforces integrity constraints (PK, FK, NOT NULL).
2. **Transaction Manager:** Manages transactions — logical units of work. Ensures the database remains consistent even after system failures. Enforces **ACID** properties. Manages the transaction log.
3. **File Manager:** Manages physical storage of data on disk. Allocates disk space, manages file organization (heap, sequential, indexed), and tracks data locations.
4. **Buffer Manager:** Manages main memory (RAM) buffer. Fetches disk pages into memory when needed and uses LRU (Least Recently Used) replacement policy to evict pages when memory is full, minimizing costly disk I/O.

Diagram:

```

[ DML Commands from Query Processor ]
|
+-----v-----+
|  STORAGE MANAGER  |
| +-----+ |
| | Auth & Integrity | |
| | Transaction Mgr  | |
| | File Manager     | |
| | Buffer Manager    | |
| +-----+ |
+-----+
|
+-----v-----+
|  Physical Disk    |
| (Data files, Indexes, |
| Logs, Data Dictionary)|
+-----+

```

Q8. Explain the Query Processor and its components in detail.

The **Query Processor** translates high-level user queries (SQL) into executable operations that the Storage Manager performs.

1. DDL Interpreter:

Interprets **CREATE**, **ALTER**, **DROP**. Validates syntax and semantics, then records schema definitions as metadata in the **Data Dictionary**. Does not return data — only modifies schema.

2. DML Compiler:

Translates **SELECT**, **INSERT**, **UPDATE**, **DELETE** into an optimized **query execution plan**:

- **Parsing:** Checks for syntax errors.
- **Query Optimization:** Finds the most efficient execution path (right index, best join order).

3. Query Evaluation Engine:

Executes the low-level plan produced by the DML compiler. Interfaces with the Storage Manager to physically fetch and process data from disk.

Query Flow:

```

User writes SQL Query
|
+----v-----+
|DDL Interp. | --> Updates Data Dictionary
+----+-----+
|
+----v-----+
|DML Compiler| --> Parses, Optimizes, Creates Query Plan
+----+-----+
|
+----v-----+
|Query Eval. | --> Executes query via Storage Manager
|Engine      |
+----+-----+
|
+----v-----+
|Storage Manager| --> Fetches from Disk
+-----+
|
Result returned to User

```

Q9. Explain the characteristics of a database system with examples.

1. **Self-Describing Nature:** A DBMS contains not just data but also a complete description (metadata) in a **Data Dictionary**, making it self-contained.
2. **Data Abstraction:** Hides physical storage details. Programmers work with logical tables, not raw files.
3. **Controlled Redundancy:** Minimizes data duplication through **normalization**, preventing inconsistency.
4. **Multiple User Interfaces:** Supports GUI forms for naive users, SQL for sophisticated users, APIs for programmers.
5. **Data Sharing:** Multiple users and applications can concurrently access the same database via **concurrency control**.
6. **Data Security:** Fine-grained privilege control (**GRANT** / **REVOKE**) prevents unauthorized access.
7. **Backup and Recovery:** Automatically logs transactions and restores the database to a consistent state after a crash.

Q10. Explain 1-tier, 2-tier, and 3-tier DBMS architecture with diagrams.

1-Tier Architecture:

All components (UI, app logic, database) on one machine. Used for personal/standalone applications.

- Pros: Simple, fast, low cost.
- Cons: Not scalable, no data sharing.
- Example: SQLite on a personal PC.

```

+-----+
|      Single Machine      |
| [UI] --> [Application] --> [Database] |
+-----+

```

2-Tier Architecture:

Client (UI + app logic) communicates directly with a Database Server over a network.

- *Pros*: Supports multiple users over LAN.
- *Cons*: Logic on client; not internet-scalable.
- *Example*: Desktop attendance app → school MySQL server.

```
[Client PC 1] ---+
[Client PC 2] ---+--> [Database Server] (LAN)
[Client PC 3] ---+
```

3-Tier Architecture:

Three independent layers — Presentation, Application Logic, and Database.

- *Pros*: Highly scalable, secure, maintainable. Industry standard.
- *Cons*: Complex to design and deploy.
- *Example*: Web browser → Node.js Server → Oracle DB.

```
[Browser/Mobile]      Tier 1: Presentation
|
[App/Web Server]      Tier 2: Business Logic
|
[Database Server]      Tier 3: Data Storage
```

Q11. Differentiate between DBMS and File System (5 points) with a table.

Feature	File System	DBMS
Data Redundancy	Very high; same data copied in multiple files	Minimal; data normalized and stored centrally
Data Inconsistency	Common; updates in one file not reflected in others	Eliminated; single update reflects system-wide
Data Access	No standard language; custom programs needed per query	Standard SQL supports flexible, ad-hoc queries
Security	Basic OS-level file permissions only	Fine-grained access control per user, table, and column
Concurrent Access	Unreliable; simultaneous writes can corrupt data	Built-in concurrency control with locking/isolation
Backup & Recovery	Manual; no automatic failure recovery	Automatic transaction logs and crash recovery
Data Integrity	Enforced manually within application code	Enforced automatically via DBMS constraints (PK, FK)
Cost	Cheap initially	Higher initial cost but lower long-term cost
Example	<code>.txt</code> , <code>.csv</code> files managed by OS	MySQL, Oracle, PostgreSQL

Q12. Explain Entity Sets and Relationship Sets in detail with examples and ER diagram notation.

1. Entity Sets

An **Entity** is an object in the real world that is distinguishable from other objects. An **Entity Set** is a set of entities of the same type that share the same properties.

* **Strong Entity Set:** Has a primary key that uniquely identifies every entity. Drawn as a **single rectangle**. Example: EMPLOYEE(Emp_ID, Name, Salary).

* **Weak Entity Set:** Lacks a primary key of its own; depends on an owner entity set. Drawn as a **double rectangle**. Example: DEPENDENT(Dep_Name, Relationship).

2. Relationship Sets

A **Relationship** is an association among several entities. A **Relationship Set** is a set of relationships of the same type.

* **Degree:** Number of participating entity sets (Binary = 2, Ternary = 3).

* **Descriptive Attributes:** Attributes belonging to the relationship set itself (e.g., Hours in WORKS_ON).

* **Notation:** Drawn as a **diamond** (or **double diamond** for identifying relationships).

```
+-----+ /\ +-----+ | EMPLOYEE |-----< >-----| PROJECT | | (Strong Entity) | \ /
| (Strong Entity) | +-----+ \/ +-----+ WORKS_ON (Relationship)
```

Q13. Explain the different types of attributes (simple, composite, single-valued, multivalued, derived) with diagrams.

```
/-----\ ( Attribute ) \-----/ | +-----+-----+
+-----+ | | | +-----v-----+ +-----v-----+ +-----v-----+ +-----v-----+ | Simple vs | | Single vs | |
Derived | | Key | | Composite | | Multi-val | | Attribute | | Attribute | +-----+ +-----+ +-----+
+-----+ +-----+
```

1. Simple vs. Composite Attributes:

* **Simple:** Atomic, non-divisible (e.g., Age, Salary).

* **Composite:** Can be divided into smaller sub-attributes (e.g., Name $\rightarrow$ First_Name, Last_Name; Address $\rightarrow$ Street, City, Zip).

2. Single-valued vs. Multivalued Attributes:

* **Single-valued:** Holds exactly one value (e.g., Date_of_Birth).

* **Multivalued:** Can hold multiple values for a single entity (e.g., Phone_Number). Drawn as a **double ellipse**.

3. Derived Attributes:

* Computed dynamically from other attributes (e.g., Age computed from Date_of_Birth). Drawn as a **dashed ellipse**.

4. Key Attribute:

* Uniquely identifies an entity in an entity set (e.g., Roll_No). Drawn with **underlined text**.

Q14. Explain Mapping Cardinality Constraints (1:1, 1:N, N:1, M:N) in detail with diagrams.

Mapping Cardinality defines the number of entities to which another entity can be associated via a relationship set:

```
1. One-to-One (1:1) 2. One-to-Many (1:N) 3. Many-to-Many (M:N) [A1] --- [B1] [A1] ---- [B1] [A1] ---- [B1]
[A2] --- [B2] +-- [B2] X [A3] --- [B3] [A2] ----- [B3] [A2] ---- [B2]
```

- **One-to-One (1:1):** Each entity in A connects to at most 1 entity in B, and vice versa. Example: EMPLOYEE *manages* DEPARTMENT.

- **One-to-Many (1:N):** An entity in A connects to multiple entities in B, but an entity in B connects to at most 1 entity in A. Example: DEPARTMENT *employs* EMPLOYEE.
- **Many-to-One (N:1):** Multiple entities in A connect to at most 1 entity in B. Example: STUDENT *belongs to* DEPARTMENT.
- **Many-to-Many (M:N):** An entity in A connects to multiple entities in B, and vice versa. Example: STUDENT *enrolls in* COURSE.

Q15. Explain Weak and Strong Entity Sets in detail with a suitable example (e.g., Section-Department) and diagram.

Comparison Table

Feature	Strong Entity Set	Weak Entity Set
Primary Key	Has its own primary key	No primary key; has a Discriminator (Partial Key)
Dependency	Independent existence	Existence-dependent on Owner Entity Set
Participation	Partial or Total	Always Total Participation in identifying relationship
ER Symbol	Single Rectangle	Double Rectangle
Relationship	Regular Relationship (Diamond)	Identifying Relationship (Double Diamond)

Example: COURSE (Strong) and SECTION (Weak)

- COURSE has primary key Course_ID.
- SECTION has discriminator Sec_Number (e.g., Sec 1, Sec 2 exist for multiple courses).
- A SECTION cannot exist without its parent COURSE.
- **Primary Key of SECTION:** (Course_ID, Sec_Number).

```
+-----+ //=====\\ +-----+ | COURSE |=====| HAS ||=====| SECTION | | (Course_ID)
| \\=====// || (Sec_Number) || +-----+ Identifying +-----+ Strong Owner Relationship
Weak Entity
```

Q16. Explain how Primary Keys are determined for entity sets and relationship sets, including all binary relationship cardinalities.

1. Primary Keys for Entity Sets

- **Strong Entity Set:** Chosen from Candidate Keys (minimal superkey).
- **Weak Entity Set:** Owner Primary Key + Weak Entity Discriminator.

2. Primary Keys for Relationship Sets

Mapping Cardinality	Primary Key of Relationship Set	Explanation
Many-to-Many (M:N)	PK(A) + PK(B) (Union of both PKs)	Every pair is unique
One-to-Many (1:N)	PK(B) (Primary Key of Many side)	Each B maps to at most 1 A
Many-to-One (N:1)	PK(A) (Primary Key of Many side)	Each A maps to at most 1 B
One-to-One (1:1)	PK(A) OR PK(B) (Either one)	Unique on both sides

Q17. Explain Specialization and Generalization in the EER model with examples and diagrams.

- **Specialization (Top-Down):** Breaking down a higher-level entity set into smaller, specialized lower-level entity sets based on distinguishing traits.
- **Generalization (Bottom-Up):** Combining multiple lower-level entity sets sharing common attributes into a single higher-level entity set.

```
+-----+ | PERSON | <-- Superclass / Higher-Level | (SSN, Name, Addr) | +-----+
/ \ (ISA Circle) / \ +-----+ +-----+ | STUDENT | | EMPLOYEE | <-- Subclasses / Lower-Level |
(GPA, Major)| | (Salary) | +-----+ +-----+
```

- **Attribute Inheritance:** Subclasses (STUDENT, EMPLOYEE) inherit all attributes (SSN, Name, Addr) from PERSON and add their own specific attributes.

Q18. Explain Total and Partial Participation constraints with examples and ER diagram notation.

- **Total Participation (Existence Dependency):** Every entity in the entity set MUST participate in at least one relationship instance. Drawn as a **Double Line**.
- **Partial Participation (Optional):** Some entities in the entity set may NOT participate in any relationship instance. Drawn as a **Single Line**.

```
+-----+ /\ +-----+ | EMPLOYEE |-----< >=====| DEPARTMENT | | (Partial) | \ / | (Total)
| +-----+ \/ +-----+ WORKS_IN (Not all Employees work in a Dept) (Every Dept MUST have
employees)
```

Q19. Explain Aggregation in the EER model with a suitable example (proj_guide, eval_for) and diagram.

Problem

In standard ER modeling, a relationship set cannot participate directly in another relationship set.

Solution

Aggregation abstracts a relationship set and its participating entity sets as a single higher-level entity set.

Example

Suppose EMPLOYEE works on PROJECT via relationship WORKS_ON. Now we need to record a MANAGER who evaluates this work via relationship EVALUATES.

We aggregate [EMPLOYEE - WORKS_ON - PROJECT] into a single unit, which then links to MANAGER via EVALUATES.

Q23. Explain the steps for mapping a regular binary M:N relationship set to a relational schema, using WORKS_ON (Employee–Project).

Mapping Rule for M:N Relationship Set

1. Create a **new separate relation (table)** for the relationship set.
2. Include the Primary Key of **both** participating entity sets as **Foreign Keys**.
3. Include any descriptive attributes of the relationship set.
4. Set the **Primary Key** of the new relation as the **combination of both Foreign Keys**.

Example: EMPLOYEE (Emp_ID) M:N PROJECT (Proj_ID) via WORKS_ON (Hours)

- EMPLOYEE table: EMPLOYEE(Emp_ID, Emp_Name, Salary) with PK = Emp_ID.
 - PROJECT table: PROJECT(Proj_ID, Proj_Name, Budget) with PK = Proj_ID.
 - WORKS_ON table: WORKS_ON(Emp_ID, Proj_ID, Hours)
 - Primary Key = (Emp_ID, Proj_ID)
 - Emp_ID references EMPLOYEE(Emp_ID)
 - Proj_ID references PROJECT(Proj_ID)
-

Q24. For the Banking System (Customer, Account, Transaction, Branch), draw the ER diagram and identify cardinalities.

Entities & Attributes

- CUSTOMER(Cust_ID, Name, Address)
- ACCOUNT(Acc_No, Balance, Acc_Type)
- BRANCH(Branch_ID, Branch_Name, City)
- TRANSACTION(Txn_ID, Date, Amount) [Weak entity dependent on ACCOUNT]

Relationships & Cardinalities

1. CUSTOMER *holds* ACCOUNT (**M:N**): A customer can hold multiple accounts (e.g., savings, current); an account can be jointly held by multiple customers.
 2. BRANCH *maintains* ACCOUNT (**1:N**): One branch maintains many accounts; each account belongs to 1 branch.
 3. ACCOUNT *has* TRANSACTION (**1:N Identifying**): One account has many transactions; each transaction belongs to 1 account.
-

Q25. Explain the mapping of weak entity sets using the DEPENDENT–EMPLOYEE example. Why is the owner's primary key included as a foreign key?

Mapping Steps

1. Create a relation for the owner entity set: EMPLOYEE(Emp_ID, Name, Salary) with PK = Emp_ID.
2. Create a relation for the weak entity set: DEPENDENT(Emp_ID, Dep_Name, Relationship, Birth_Date).
3. Include the Primary Key of the owner (Emp_ID) as a **Foreign Key** in DEPENDENT.
4. Set the **Primary Key of DEPENDENT** = (Emp_ID, Dep_Name) (Owner PK + Discriminator).

Why Owner's Primary Key is Included

A weak entity set lacks a primary key of its own (e.g., Dep_Name like "John" is not globally unique). Including Emp_ID grounds the weak entity to its specific owner, creating a composite primary key (Emp_ID, Dep_Name) that uniquely identifies each dependent record in the database.

Q26. Describe the mapping of a binary 1:N relationship set with an example, showing foreign key placement on the "N" side.

Mapping Rule for 1:N Relationship

Do **NOT** create a separate table. Instead:

1. Identify the relation on the "**N**" (**Many**) side.
2. Add the Primary Key of the "**1**" (**One**) side as a **Foreign Key** inside the "N" side table.
3. Add any descriptive attributes of the relationship into the "N" side table.

Example: DEPARTMENT (1) *employs* EMPLOYEE (N)

- DEPARTMENT(Dept_ID, Dept_Name) with PK = Dept_ID.
 - EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID)
 - PK = Emp_ID
 - Dept_ID is a **Foreign Key** referencing DEPARTMENT(Dept_ID).
-

Q27. Explain N-ary relationship mapping using the SUPPLIER-PROJECT-PART (SUPPLY) example. Why can't it be modeled as binary?

N-ary Relationship Mapping Rule

Create a **new relation** for the N-ary relationship containing:

1. Primary Keys of **all participating entity sets** as Foreign Keys.
2. Descriptive attributes of the relationship.
3. Primary Key = Combination of participating Primary Keys.

Example: SUPPLY (Degree 3 / Ternary)

- SUPPLIER(S_ID), PROJECT(P_ID), PART(Part_ID).
- SUPPLY(S_ID, P_ID, Part_ID, Quantity) with PK = (S_ID, P_ID, Part_ID).

Why It Cannot Be Modeled as Binary

3 binary relationships (Supplier-Part, Supplier-Project, Part-Project) only state that Supplier S sells Part P, Supplier S works on Project PR, and Part P is used in Project PR. They **cannot assert** that *Supplier S supplied Part P specifically for Project PR in a single atomic transaction*.

Q28. For the College Admission System, design the ER diagram and convert it into a relational schema (relations with keys).

ER Components

- **Entities:** APPLICANT(App_ID, Name, Marks), COURSE(Course_ID, Title, Capacity).
- **Relationship:** APPLIES_FOR(App_ID, Course_ID, Application_Date, Status) (M:N relationship).

Converted Relational Schema

1. APPLICANT (App_ID, Name, Marks)
2. COURSE (Course_ID, Title, Capacity)
3. APPLIES_FOR (App_ID, Course_ID, Application_Date, Status)
 - * App_ID references APPLICANT(App_ID)
 - * Course_ID references COURSE(Course_ID)

Q29. Compare the four approaches to mapping specialization/generalization, stating when each applies.

Approach	Relational Schema Structure	Primary Key	Best Applied When
1. Superclass + Subclasses	1 table for superclass + 1 table per subclass	Superclass PK is PK for all tables	Generalization is overlapping/disjoint with many specific attributes
2. Subclass-Only	Separate tables for each subclass only (no superclass table)	Subclass PK	Generalization is Total and Disjoint
3. Single Relation (1 Type Attr)	1 unified table with a Type discriminator column	Superclass PK	Specialization is Disjoint with few specific attributes
4. Single Relation (Multi Type Attrs)	1 unified table with boolean flags (is_sec, is_eng)	Superclass PK	Specialization is Overlapping with few specific attributes